



DIGITAL IMAGE PROCESSING

LAB 06

M JAWAD RAZA

235119

Project Title:

Smart Image Enhancement & Analysis System



1. Objective

The main goal of this project is to develop a user-friendly system for enhancing and analyzing images. The system allows users to:

- Load and view images.
- Apply intensity transformations like negative, logarithmic, and gamma correction.
- Perform geometric transformations including rotation, translation, and shearing.
- Carry out sampling, quantization, and histogram equalization to improve image quality.
- Analyze the image by displaying resolution, data type, and sample pixel values.
- Save the processed images and visualize their histograms.

This system is designed to provide an interactive GUI-based interface that makes image processing easy for both learning and practical applications.

2. Methodology

The development of this system was carried out in several phases:

Phase 1: GUI Design

The graphical user interface (GUI) was built using Tkinter.

- A gradient background was applied for visual appeal.
- A control panel was added on the left side with buttons, checkboxes, and dropdown menus for different processing options.
- An analysis panel was placed to show image information dynamically.

Phase 2: Loading Images

- Users can load an image through a “Load Image” button.

- Images are automatically converted to grayscale for processing, while the original color image is kept for reference.
- The original and processed images are displayed in the **bottom-right corner** of the GUI.

Phase 3: Image Enhancement

- **Intensity Transformations:** Negative image, log transformation, gamma correction.
- **Geometric Transformations:** Rotation (with inverse option), translation (with inverse), shearing (with inverse).
- **Sampling & Quantization:** Resize images and reduce the number of gray levels.
- **Histogram Equalization:** Enhances image contrast for better visualization.

Phase 4: Analysis and Output

- The analysis panel updates in real-time with resolution, data type, and a sample of pixel values.
- Users can view the histogram of the processed image to analyze intensity distribution.
- Processed images can be saved with a single click.

3. Code Snippets

GUI Setup and Gradient Background

```
# Main window
```

```
root = Tk()
```

```
root.title("Smart Image Enhancement System")
```

```
root.state("zoomed")
```

```
# Canvas for gradient and panels

canvas = Canvas(root)

canvas.pack(fill="both", expand=True)

canvas.bind("<Configure>", draw_gradient)


# Gradient background function

def draw_gradient(event=None):

    canvas.delete("all")

    width = canvas.winfo_width()

    height = canvas.winfo_height()

    for i in range(height):

        r = int(100 + (i / height) * 100)

        g = int(150 + (i / height) * 80)

        b = 255

        color = f'#{r:02x}{g:02x}{b:02x}'

        canvas.create_line(0, i, width, i, fill=color)
```

Image Loading and Display

```
def load_image():

    global original, gray, processed

    path = filedialog.askopenfilename()

    if not path:
```

```
        return

original = cv2.imread(path)

gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY)

processed = gray.copy()

display(original, panelA)

display(processed, panelB)


def display(img, panel, max_size=(320, 320)):

    if img is None:

        return

    if len(img.shape) == 2: # grayscale to RGB

        img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)

    else:

        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    img = Image.fromarray(img)

    w, h = img.size

    max_w, max_h = max_size

    ratio = min(max_w/w, max_h/h)

    img = img.resize((int(w*ratio), int(h*ratio)))

    img = ImageTk.PhotoImage(img)

    panel.config(image=img)

    panel.image = img
```

```
# Image panels (bottom-right)
```

```
panelA = Label(canvas)
```

```
panelA.place(relx=0.98, rely=0.98, anchor="se") # Original
```

```
panelB = Label(canvas)
```

```
panelB.place(relx=0.75, rely=0.98, anchor="se") # Processed
```

Image Analysis Panel

```
analysis_text = Text(canvas, width=40, height=10, font=("Arial",11))
```

```
analysis_text.place(relx=0.02, rely=0.75)
```

```
analysis_text.config(state=DISABLED)
```

```
def update_analysis():
```

```
    if gray is None:
```

```
        return
```

```
    info = f"Resolution: {gray.shape[1]}x{gray.shape[0]}\n"
```

```
    info += f"Data type: {gray.dtype}\n"
```

```
    info += f"Sample matrix (5x5 top-left):\n{gray[:5,:5]}\n"
```

```
    analysis_text.config(state=NORMAL)
```

```
    analysis_text.delete("1.0", END)
```

```
    analysis_text.insert(END, info)
```

```
    analysis_text.config(state=DISABLED)
```

Intensity Transformations

Negative

if negative_var.get():

 processed = 255 - processed

Log transformation

if log_var.get():

 c = 255 / np.log(1 + np.max(processed))

 processed = c * np.log(1 + processed)

 processed = np.uint8(processed)

Gamma correction

if gamma_var.get():

 g = float(gamma_value.get())

 norm = processed / 255.0

 processed = np.power(norm, g)

 processed = np.uint8(processed * 255)

Geometric Transformations

Rotation

if rotate_var.get():

```
angle = int(angle_var.get())
```

```
h, w = processed.shape
```

```
M = cv2.getRotationMatrix2D((w//2, h//2), angle, 1)
```

```
processed = cv2.warpAffine(processed, M, (w, h))
```

```
if inverse_rotate_var.get():
```

```
    M_inv = cv2.getRotationMatrix2D((w//2, h//2), -angle, 1)
```

```
    processed = cv2.warpAffine(processed, M_inv, (w, h))
```

```
# Translation
```

```
if translate_var.get():
```

```
    tx = int(tx_var.get())
```

```
    ty = int(ty_var.get())
```

```
M = np.float32([[1,0,tx],[0,1,ty]])
```

```
processed = cv2.warpAffine(processed, M, (processed.shape[1], processed.shape[0]))
```

```
if inverse_translate_var.get():
```

```
    M_inv = np.float32([[1,0,-tx],[0,1,-ty]])
```

```
    processed = cv2.warpAffine(processed, M_inv, (processed.shape[1], processed.shape[0]))
```

```
# Shearing
```

```
if shear_var.get():
```

```
    shx = float(shx_var.get())
```

```
    shy = float(shy_var.get())
```



```
M = np.float32([[1, shx, 0],[shy,1,0]])

processed = cv2.warpAffine(processed, M, (processed.shape[1], processed.shape[0]))

if inverse_shear_var.get():

    M_inv = np.float32([[1,-shx,0],[-shy,1,0]])

    processed = cv2.warpAffine(processed, M_inv, (processed.shape[1], processed.shape[0]))
```

Sampling & Quantization

Sampling

```
if sample_var.get():

    scale = float(scale_var.get())

    processed = cv2.resize(processed, None, fx=scale, fy=scale, interpolation=cv2.INTER_LINEAR)
```

Quantization

```
if quant_var.get():

    bits = int(bit_var.get())

    levels = 2 ** bits

    step = 256 // levels

    processed = (processed // step) * step
```

Histogram Equalization & Visualization

Histogram equalization

```
if hist_var.get():

    processed = cv2.equalizeHist(processed)
```

```
# Display histogram

def show_histogram():

    if processed is None:

        return

    plt.figure("Histogram")

    plt.hist(processed.ravel(), bins=256, color='gray')

    plt.title("Histogram")

    plt.show()
```

Save Processed Image

```
def save_image():

    if processed is None:

        return

    cv2.imwrite("enhanced_output.jpg", processed)

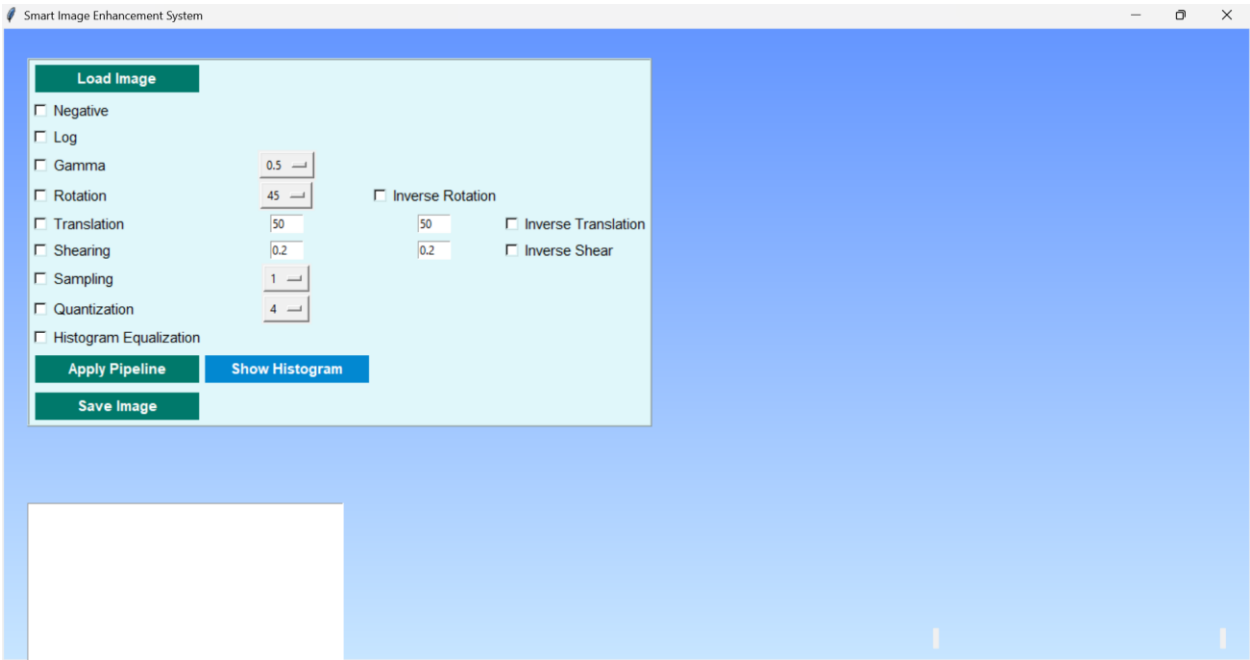
    messagebox.showinfo("Saved", "Image saved as 'enhanced_output.jpg'!")
```

Complete Pipeline Button

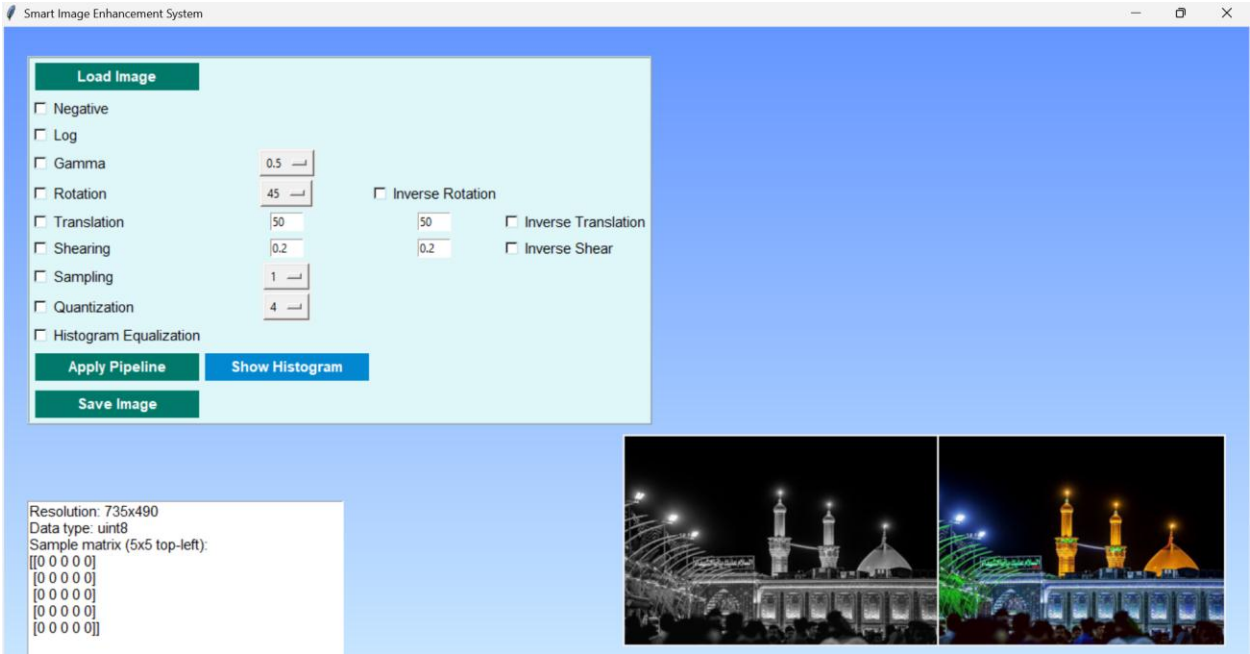
```
Button(frame, text="Apply Pipeline", command=apply_pipeline,
**btn_main).grid(row=10,column=0,pady=5)
```

4. Input/Output Images

Original Image:



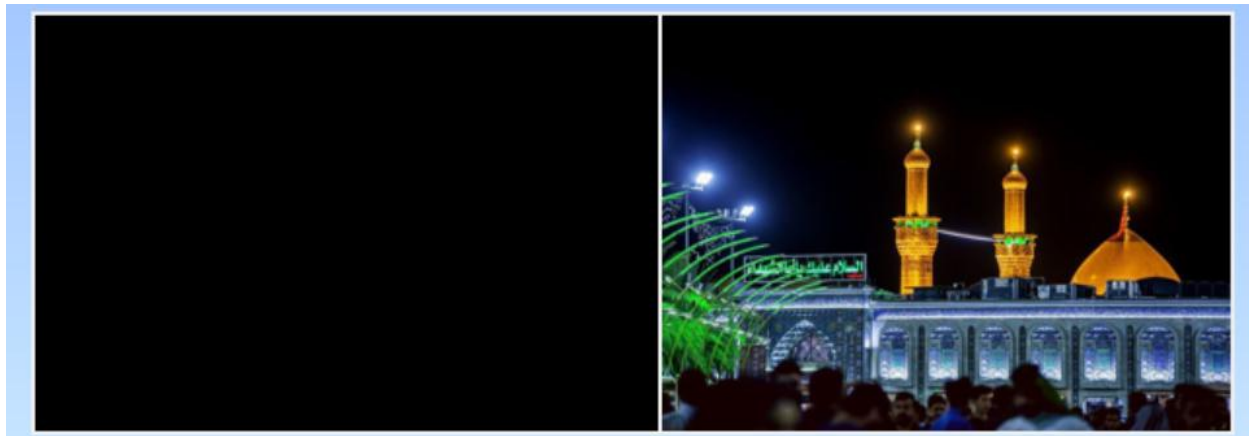
Processed Image:



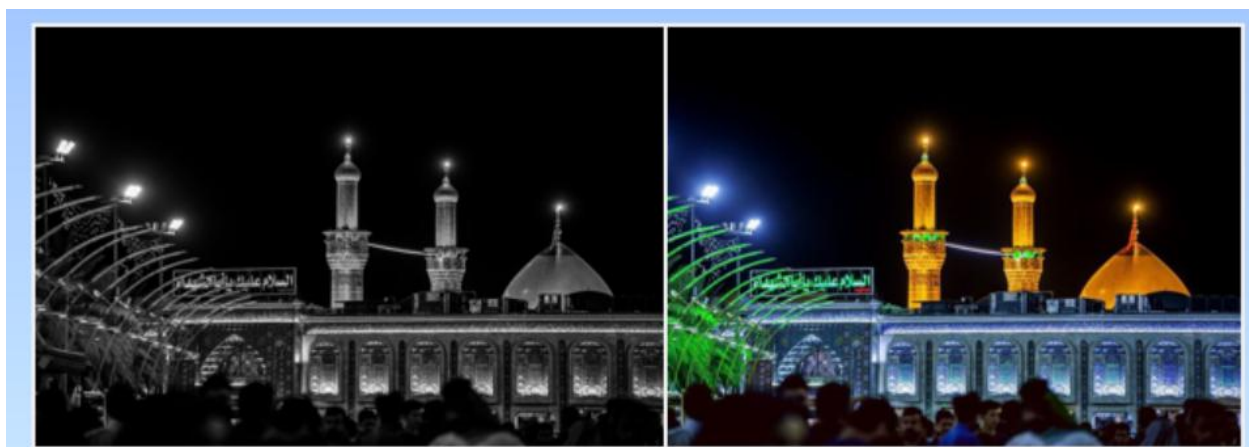
Negative:



Log:



Gamma:



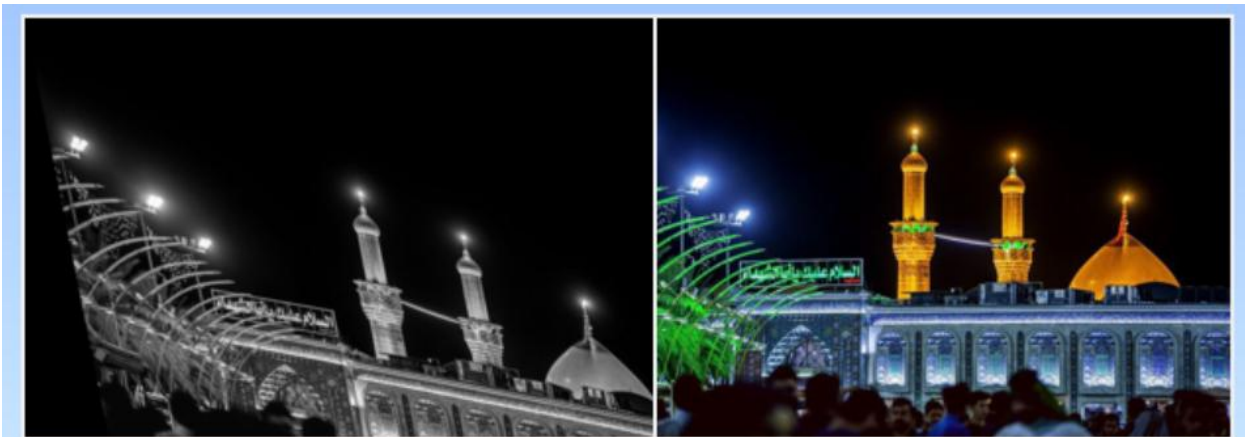
Rotation 45:



Translation:



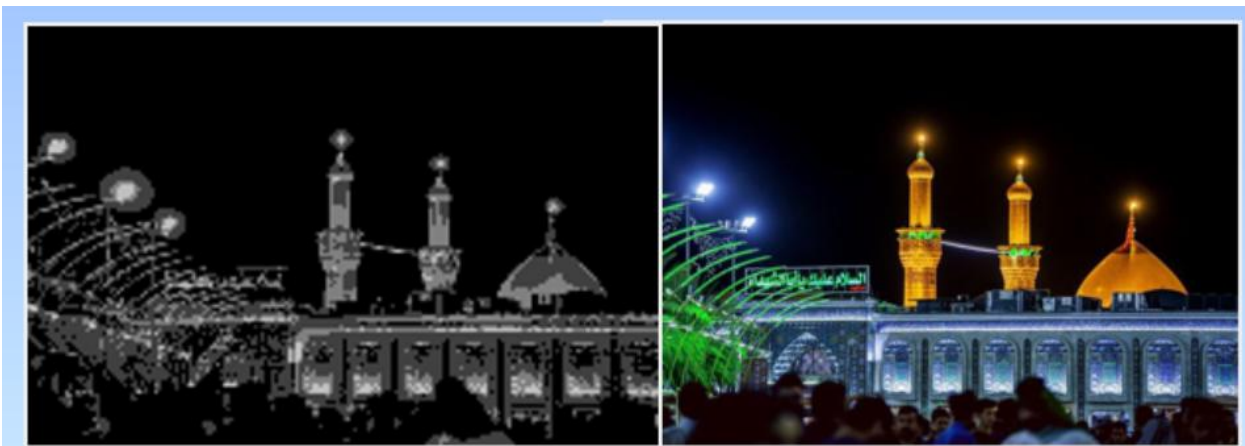
Shearing:



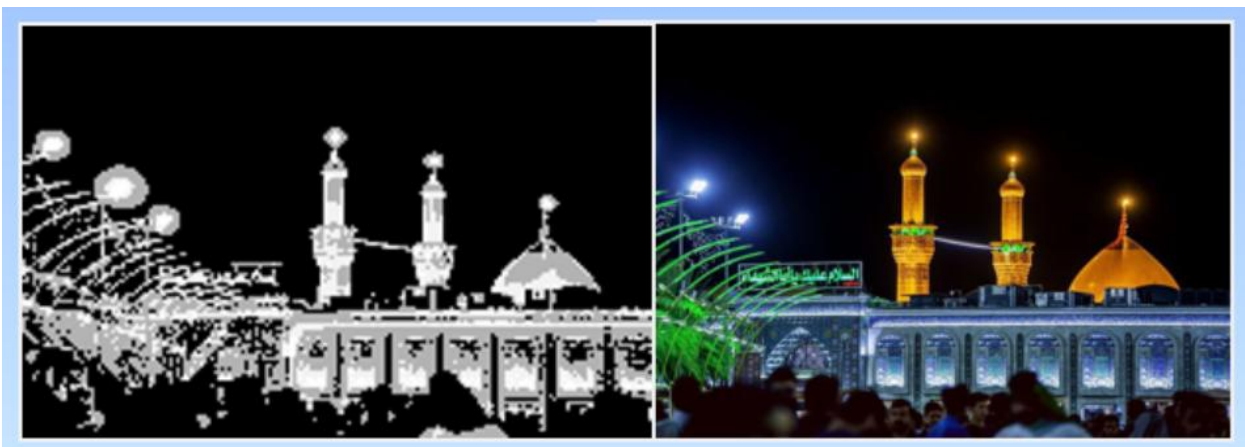
Sampling 0.5:



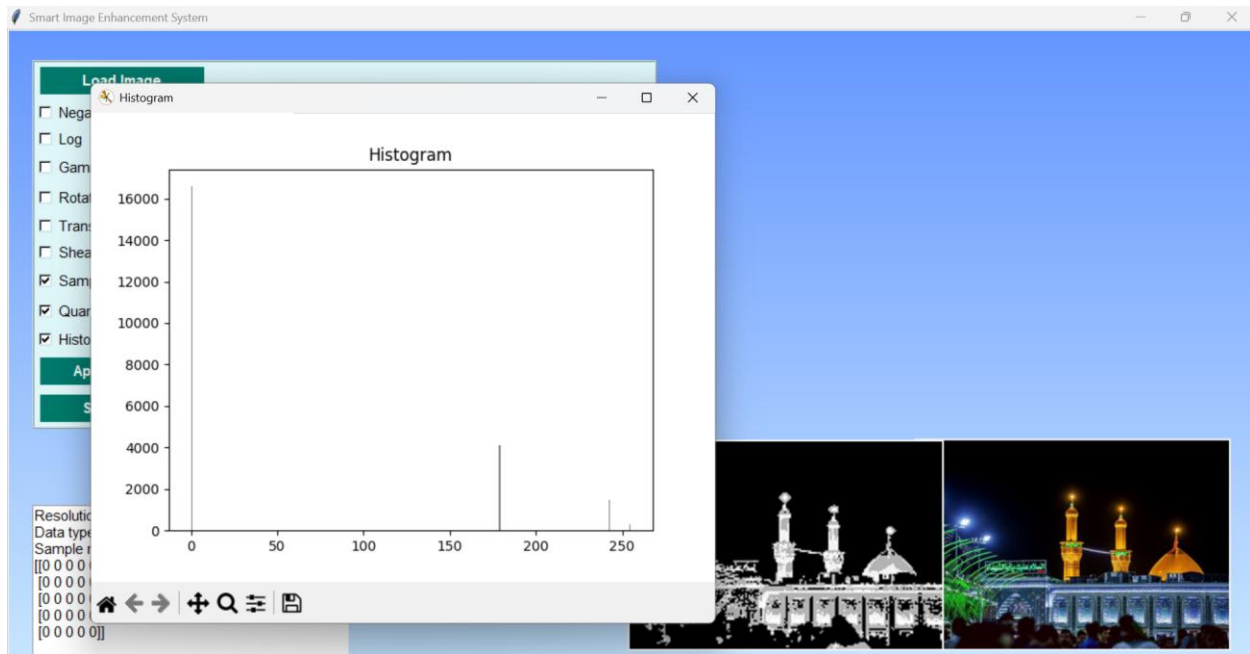
Quantization 2:



Histogram Equalization:



Histogram Visualization:



5. Observations and Analysis

- **Negative Transformation:** Inverts pixel values, producing a photographic negative.
- **Gamma Correction:** Adjusts image brightness. $\text{Gamma} < 1$ brightens the image; $\text{gamma} > 1$ darkens it.
- **Log Transformation:** Expands dark pixel values, enhancing low-light areas.
- **Rotation:** Rotates the image around its center. Using inverse rotation restores the original orientation.
- **Translation:** Moves the image horizontally or vertically. Inverse translation returns the image to its original position.
- **Shearing:** Skews the image along X and Y axes. Using inverse shear corrects distortions.
- **Sampling:** Resizes the image to reduce or increase detail.
- **Quantization:** Reduces the number of gray levels, which may create a posterization effect.

- **Histogram Equalization:** Improves image contrast by redistributing pixel intensities.

6. Conclusion

The Smart Image Enhancement System successfully provides an interactive platform for image processing.

- Users can perform both intensity and geometric transformations in real-time.
- The analysis panel and histogram visualization offer insight into pixel distributions.
- Images are displayed clearly in the bottom-right corner, separated from control elements.
- The system is user-friendly, flexible, and visually appealing, meeting all lab project requirements.

This project demonstrates effective integration of Tkinter GUI, OpenCV image processing, and Python programming to create a practical image enhancement tool suitable for academic and learning purposes.

7. Question / Answer

1. Why does histogram equalization improve contrast?

Histogram equalization redistributes the intensity values of an image so that they span the full range of possible values. This stretches out areas of low contrast and compresses areas of high intensity concentration, making details more visible. In other words, it spreads out the pixel values to use the entire grayscale range, which enhances visual contrast.

2. How does gamma affect brightness?

Gamma correction adjusts the luminance of an image non-linearly. If gamma is less than 1, the image becomes brighter because darker pixels are amplified. If gamma is greater than 1, the image becomes darker because brighter pixels are suppressed. This allows fine-tuning of brightness without changing the extremes too drastically.

3. What is the effect of quantization on image quality?

Quantization reduces the number of gray levels in an image. While it decreases the data size and simplifies intensity values, it can introduce a “posterization” effect where smooth gradients

appear as distinct bands. High quantization levels preserve quality, but aggressive quantization reduces visual detail and smoothness.

4. Which transformation is reversible and why?

Transformations like rotation, translation, and shearing (when not clipped) are reversible because they are geometric operations that can be undone using the inverse matrix. For example, an image rotated by 45° can be restored using a -45° rotation. However, transformations like quantization or gamma correction are generally not perfectly reversible, as some pixel information is lost during processing.

5. How do transformations affect spatial structure?

Geometric transformations (rotation, translation, shearing) change the positions of pixels, altering the spatial structure of the image. While intensity transformations (negative, gamma, log) do not move pixels, they modify pixel values, which can affect contrast and visibility of features. Combining transformations allows both spatial and intensity changes, which can enhance or distort the original image depending on how they are applied.